Práctica. Análisis estático de código con Bandit

Nombre y apellidos: _____

Martes 10 de noviembre de 2026 · PPS · 4 horas · trabajo individual

La semana pasada las pruebas encontraron los fallos porque vosotros sabíais qué buscar: teníais una especificación. Hoy vamos a usar una herramienta que lee el código buscando patrones peligrosos sin saber qué tiene que hacer el programa. Se llama Bandit, es de las más usadas para Python y la vais a encontrar en muchas empresas.

Bandit es rápido y encuentra cosas reales, pero también se equivoca. Os va a dar avisos que son fallos de verdad, avisos que no lo son y avisos que solo os piden que miréis con atención. Y hay fallos que no va a ver. Vuestro trabajo hoy no es ejecutar Bandit, eso lo hace cualquiera en diez segundos. Vuestro trabajo es decidir qué hacer con cada aviso y saber explicar por qué.

1. Qué vas a aprender hoy

Al terminar la sesión tienes que ser capaz de:

1. Ejecutar Bandit sobre un proyecto y limitar lo que analiza.
2. Leer un aviso: identificador, severidad, confianza, CWE y ubicación.
3. Clasificar cada aviso como verdadero positivo, falso positivo o informativo, mirando el código y no solo el mensaje.
4. Corregir los verdaderos positivos con una prueba de regresión que falle antes y pase después.
5. Marcar un falso positivo con `# nosec` de forma justificada y concreta.
6. Configurar Bandit en `pyproject.toml` y comparar el antes y el después.
7. Explicar qué tipo de fallos no puede ver una herramienta de análisis estático.

2. Lo que necesitas

Conocimientos: la práctica de pytest del 3 de noviembre (entorno virtual, `parametrize`, `pytest.raises`, fixtures). Si no la terminaste, mira el itinerario de recuperación del apartado 12.

Equipo: tu Ubuntu del aula o tu portátil con Python 3.10 o superior. No necesitas Docker ni Git.

Herramientas: `python3`, `venv`, `pip`, `pytest` y `bandit`.

3. Prepara el entorno (15 minutos)

Descarga `PPS-2026-11-10-bandit-LAB.zip` de la web del curso (PPS, Prácticas) y descomprímelo. Si tu carpeta de descargas o de trabajo se llama de otra forma, ajusta las rutas:

```
mkdir -p ~/pps
cd ~/pps
unzip ~/Descargas/PPS-2026-11-10-bandit-LAB.zip
cd PPS-2026-11-10-bandit-LAB
ls
```

Tienes que ver `README.md`, `informe.md`, `experimento_assert.py`, `pyproject.toml` y las carpetas `reservas`, `tests` e `informes`. Es una aplicación pequeña para reservar el aula de informática. No tiene interfaz web: son las funciones que usaría una web por detrás, y en los comentarios te digo de dónde llega cada dato.

Crea el entorno e instala lo necesario:

```
python3 -m venv .venv
. .venv/bin/activate
pip install pytest "bandit[toml]"
```

El `[toml]` entre corchetes instala el soporte opcional para que Bandit pueda leer su configuración desde `pyproject.toml`; con Python 3.11 o superior ya viene en la biblioteca estándar, pero con 3.10 hace falta. Las comillas son necesarias para que la terminal no interprete los corchetes.

Comprueba que todo funciona:

```
python -m pytest
bandit --version
```

pytest tiene que decir `8 passed`. Apunta en `informe.md` la versión de Bandit que te sale; los avisos pueden variar un poco de una versión a otra.

4. Primera pasada: cuidado con lo que analizas (15 minutos)

Lo primero que hace todo el mundo es esto:

```
bandit -r .
```

`-r` significa recursivo: analiza la carpeta y todo lo que hay dentro. Mira el resumen del final, en `Run metrics`. Te van a salir **cientos** de avisos. Apunta el número.

¿De verdad tiene tantos fallos una aplicación de cuatro ficheros? No. Mira la ruta de los avisos (`Location:`). La mayoría están dentro de `.venv`: Bandit ha analizado el código de pip, de pytest y de él mismo. Ese código no es tuyo y no lo vas a corregir. Los demás están en `tests`, y son casi todos el mismo aviso: `B101`, uso de `assert`. En las pruebas, `assert` es justo lo que tiene que haber.

Ahora analiza solo tu código:

```
bandit -r reservas
```

Apunta el número de avisos. Este es el **informe inicial**, el que vas a usar toda la mañana.

Lo que acabas de ver pasa en las empresas: si el informe tiene cuatrocientos avisos, nadie lo lee y los cinco importantes se pierden entre el ruido. Limitar bien lo que se analiza es la primera decisión, no un detalle.

5. Aprende a leer un aviso (15 minutos)

Cada aviso tiene esta forma:

```
>> Issue: [B602:subprocess_popen_with_shell_equals_true]
   subprocess call with shell=True identified, security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   Location: reservas/utilidades.py:32:16
```

- `B602` es el identificador de la regla. Lo necesitarás para buscarla en la documentación y para marcar falsos positivos.
- `Severity` es lo grave que sería si el aviso es cierto.
- `Confidence` es lo seguro que está Bandit de que el aviso es cierto. No es lo mismo que la severidad: un aviso puede ser muy grave y Bandit estar poco seguro.
- `CWE` es el tipo de debilidad en el catálogo CWE de MITRE, el mismo que usan los informes de auditoría profesionales. Abre el enlace del CWE-78 y lee el resumen.
- `Location` es fichero, línea y columna.

Bandit también te dice algo cuando termina. Ejecuta esto justo después de analizar:

```
bandit -r reservas > /dev/null
echo $?
```

`echo $?` muestra el código de salida de la última orden. Bandit devuelve `1` si encuentra algún aviso y `0` si no encuentra ninguno. Gracias a eso, un script o un sistema de integración continua puede parar automáticamente cuando aparece un fallo nuevo.

6. Triaje: decide qué es cada aviso (45 minutos)

Esta es la parte importante de la práctica. Abre `informe.md` y rellena la tabla del apartado 2 con una fila por cada aviso del informe inicial. Para cada uno tienes que abrir el fichero, ir a la línea y leer la función entera, no solo la línea marcada. Después decides:

- **VP, verdadero positivo:** es un fallo de seguridad real y hay que corregirlo.
- **FP, falso positivo:** Bandit ha visto un patrón que se parece a un fallo, pero en este código no lo es. Tienes que poder explicar por qué.
- **INF, informativo:** no es un fallo por sí solo, pero te avisa de algo que conviene vigilar. Por ejemplo, que se importa un módulo con el que es fácil equivocarse.

Para decidir, hazte siempre la misma pregunta: **¿de dónde viene el dato?** Si llega del usuario (un formulario, una URL, una cookie), desconfía. Si es un valor fijo del programa, el riesgo cambia. Los comentarios del código te dicen de dónde llega cada dato.

Algunas pistas, sin darte la respuesta.

Pista 1. Hay dos avisos `B608` de SQL. No son iguales. Mira qué valores puede tomar la variable que se mete en cada consulta.

Pista 2. Hay dos avisos `B105` de contraseñas escritas en el código. Lee qué guarda cada variable.

Pista 3. Para el aviso `B101` del código de la aplicación (no el de las pruebas), haz este experimento desde la carpeta del proyecto. El script `experimento_assert.py` intenta borrar una reserva sin ser administrador:

```
python experimento_assert.py
python -O experimento_assert.py
```

`-O` es la opción de Python para optimizar. Cualquier programa puede acabar ejecutándose así, y entonces los `assert` desaparecen. Compara las dos salidas y explica en el informe qué ha pasado.

Pista 4. El aviso `B311` habla del generador de números aleatorios. Lee para qué se usa ese número. Por cierto, en el mismo fichero hay otra función que usa `random` y Bandit no dice nada de ella. ¿Es un problema?

No corrijas nada todavía. Primero termina la tabla completa.

7. Un filtro peligroso (10 minutos)

Bandit permite ocultar los avisos menos graves o menos seguros. `-ll` deja solo severidad media o alta, y `-ii`, solo confianza media o alta. Prueba:

```
bandit -r reservas -ll
bandit -r reservas -ll -ii
```

Compara los dos resultados con tu tabla de triaje. ¿Ha desaparecido algún verdadero positivo con el segundo filtro? ¿Cuál y por qué? Apúntalo, porque es una de las preguntas del apartado 13.

8. Corrige los verdaderos positivos (70 minutos)

Para cada aviso que hayas clasificado como VP vas a seguir siempre los mismos tres pasos, que ya conoces de la práctica anterior:

1. Escribe en `tests/test_regresion.py` una prueba que demuestre el fallo. Ejecútala y comprueba que **falla**.
2. Corrige el código.
3. Ejecuta todas las pruebas. La nueva tiene que pasar y las 8 que ya había, también. No las modifiques: el uso normal de cada función tiene que seguir funcionando igual.

Las pruebas que ya hay están escritas para que sigan pasando si la corrección es buena. Si una corrección rompe alguna, revísala antes de seguir.

Te dejo una orientación para cada tipo de fallo. Busca los detalles en la documentación enlazada en las referencias.

Consultas SQL construidas con texto. En el mismo fichero tienes una función que hace bien la inserción con marcadores `?`. Úsala de modelo. Una prueba de regresión razonable: buscar las reservas de un usuario que se llama `x' OR '1'='1` y comprobar que no devuelve las de los demás.

Contraseñas guardadas con MD5. MD5 es muy rápido y no lleva sal, así que dos personas con la misma contraseña tienen el mismo hash y un atacante puede probar miles de millones por segundo. La biblioteca estándar tiene `hashlib.scrypt`, que es lento a propósito y admite una sal aleatoria. La sal tiene que guardarse junto al hash, y para comparar usa `hmac.compare_digest`. Prueba de regresión: el hash de la misma contraseña calculado dos veces tiene que salir distinto y, aun así, `verificar_password` tiene que aceptar la contraseña correcta y rechazar una incorrecta con cada uno de los dos hashes. Si no puedes verificar, es que no has guardado bien la sal.

Token de recuperación con el módulo random. El módulo `random` sirve para simulaciones y juegos, pero su secuencia se puede predecir. Para todo lo que tenga que ver con seguridad, Python tiene el módulo `secrets`.

Sesión guardada con pickle. `pickle.loads` puede ejecutar código al cargar los datos, y aquí los datos son una cookie que controla el usuario. Aunque no ejecutara nada, cualquiera podría fabricarse una cookie con `"rol": "admin"`. Cambia el formato a JSON. Prueba de regresión: crear con `pickle` una cookie falsa y comprobar que `leer_cookie` la rechaza con un error. Ojo: con JSON ya no se ejecuta código al leer la cookie, pero el usuario sigue pudiendo escribir `"rol": "admin"` en ella. Lo que diga una cookie no se puede creer sin más; en una aplicación de verdad la cookie va firmada o solo guarda un identificador, y los permisos se comprueban en el servidor. Explica esto en el informe; no hace falta que lo programes.

Comando ejecutado a través de la terminal. Con `shell=True`, la terminal interpreta toda la cadena, incluidos caracteres como `;`. Compruébalo desde la carpeta del proyecto. Primero mira que no existe ningún fichero `inyectado.txt` (`ls inyectado.txt` tiene que decir que no existe) y después:

```
python -c "from reservas import utilidades
utilidades.comprobar_equipo('127.0.0.1; touch inyectado.txt')"
ls -l inyectado.txt
```

Si ahora el fichero existe, el `touch` se ha ejecutado, y podría haber sido cualquier otra orden. Bórralo con `rm inyectado.txt`. Para corregirlo, pasa el comando como una lista de argumentos y quita `shell=True`. Además, comprueba antes que lo que llega es una IP de verdad: el módulo `ipaddress` de la biblioteca estándar lanza `ValueError` si no lo es.

Un assert como control de acceso. Sustitúyelo por un `if` que lance una excepción de verdad, por ejemplo `PermissionError`, y repite el experimento con `python -O experimento_assert.py`.

Contraseña del administrador en el código. Cualquiera que vea el código, o una copia antigua, la conoce. Sácala del código y léela de una variable de entorno con `os.environ.get`. Si la variable no existe, `os.environ.get` devuelve `None`: asegúrate de que en ese caso nadie puede entrar como administrador, y no pongas una contraseña por defecto. El 17 de noviembre trabajaremos a fondo los secretos y la configuración; hoy basta con que no esté escrita en el fichero.

8.1. Los falsos positivos se marcan, no se ignoran

Cuando vuelvas a ejecutar Bandit, los falsos positivos seguirán ahí. Si los dejas, el informe nunca llegará a cero y la semana que viene nadie lo mirará. Bandit permite marcar una línea para que no la avise:

```
CAMPO_EJEMPLO = "token"  # nosec B105
```

Tres reglas, y las tres cuentan en la nota:

1. Escribe siempre el identificador (`# nosec B105`), nunca `# nosec` a secas. Sin identificador, la línea deja de comprobarse para todas las reglas, también para las que todavía no sabes que existen.
2. Pon encima un comentario corto que explique por qué no es un fallo. Quien lea el código dentro de un año tiene que entenderlo sin preguntarte.
3. Solo se marcan falsos positivos. Marcar un verdadero positivo para que el informe salga limpio es esconder un fallo, y en una auditoría se considera mucho más grave que el fallo en sí.

Si en tu corrección de `shell=True` Bandit te da avisos nuevos (`B603`, `B607`), léelos. Son avisos de baja severidad que te recuerdan que sigues ejecutando un programa externo; no son falsos positivos automáticamente. Solo puedes marcarlos si has quitado `shell=True`, pasas los argumentos como lista, el programa y sus opciones son fijos y el único dato del usuario está validado. Justifica cada identificador por separado en el comentario.

9. Deja Bandit configurado en el proyecto (15 minutos)

Para no tener que acordarte de qué carpetas excluir, añade esto al final de `pyproject.toml`:

```
[tool.bandit]
exclude_dirs = [".venv", "tests"]
```

Y ejecútalo así, desde la carpeta del proyecto:

```
bandit -c pyproject.toml -r .
```

Comprueba que te salen los mismos avisos que con `bandit -r reservas`. Ahora Bandit analiza también `experimento_assert.py` y cualquier fichero `.py` que añadas fuera de `tests`, y eso es lo que queremos. Ahora cualquiera que descargue el proyecto analiza lo mismo que tú.

Guarda los informes de antes y de después para la entrega. Para el de antes, usa el código original: descomprime otra vez el ZIP en otra carpeta, por ejemplo `~/pps/original`, y desde allí ejecuta:

```
bandit -r reservas -f txt -o ~/pps/PPS-2026-11-10-bandit-LAB/bandit-antes.txt
```

Y en tu proyecto corregido:

```
bandit -c pyproject.toml -r . -f txt -o bandit-despues.txt
```

`-f` elige el formato y `-o` el fichero de salida. Hay más formatos, como `json`, `csv` o `html`. Si en algún momento quieres usar un informe como línea base (ampliación 1), tiene que ser en `json`.

10. Lo que Bandit no ve (20 minutos)

Bandit busca patrones conocidos en cada línea, pero no entiende para qué sirve el programa. Hay un fallo en `reservas/informes.py` que Bandit no marca. Lee la función, fíjate en de dónde llega `nombre` y prueba esto:

```
python -c "from reservas import informes;
print(informes.leer_informe('ocupacion-octubre.txt'))"
python -c "from reservas import informes;
print(informes.leer_informe('../pyproject.toml'))"
```

La primera orden es el uso normal. Explica qué hace la segunda y qué podría leer alguien que tuviera acceso a esa URL en un servidor de verdad. Busca cómo se llama este tipo de fallo y su número de CWE.

Corrígelo con el mismo método: primero una prueba con `parametrize` que use dos o tres nombres maliciosos y espere `PermissionError`, después el código. Una pista: `Path.resolve()` convierte una ruta en absoluta y resuelve los `..`, y `Path.is_relative_to()` te dice si una ruta está dentro de otra. Resuelve las dos, la carpeta de informes y la ruta pedida, y compara las versiones absolutas. Y comprueba que `leer_informe('ocupacion-octubre.txt')` sigue funcionando: la prueba que ya había lo vigila.

Apunta en el informe por qué crees que Bandit no puede detectar este fallo y qué otra técnica lo habría encontrado.

11. Si algo no funciona

`bandit: command not found`: no has activado el entorno, o lo has instalado en otro. Ejecuta `. .venv/bin/activate` desde la carpeta del proyecto.

`error: externally-managed-environment`: estás usando el pip del sistema. Activa el entorno y repite.

Bandit no hace caso a `pyproject.toml`: tienes que pasarle `-c pyproject.toml`; no lo lee por su cuenta. Si te dice que no puede leer el fichero, instala el extra con `pip install "bandit[toml]"`.

Un `# nosec` no tiene efecto: comprueba que está en la misma línea que marca el aviso. En las órdenes que ocupan varias líneas, el aviso aparece en la línea donde empieza la llamada.

`ModuleNotFoundError: No module named 'reservas'`: estás en otra carpeta. Vuelve a la del proyecto, donde está `pyproject.toml`.

Una prueba de regresión no falla con el código original: entonces no demuestra el fallo. Revisa que el dato de prueba llega de verdad a la línea problemática.

Sin Internet en el aula: en Ubuntu puedes instalar las herramientas con `sudo apt install bandit python3-pytest` y usarlas sin entorno virtual. Ten en cuenta que en Ubuntu 24.04 ese paquete es Bandit 1.6.2, bastante más antiguo que el de pip: el número de avisos, algunos mensajes y la lectura de `pyproject.toml` pueden cambiar, así que no te saldrá exactamente lo que dice esta hoja. Anota la versión en el informe y, en cuanto tengas Internet, repite el análisis con el entorno virtual.

12. Elige tu itinerario

Recuperación

Si no terminaste la práctica de pytest del 3 de noviembre, haz primero las reglas R1 y R4 de aquella hoja. Después, en esta, haz como mínimo los apartados 4 a 6 y corrige la inyección SQL y la sesión con `pickle`, cada una con su prueba de regresión. Tu entrega es la misma del apartado 14, pero con esas dos correcciones; en `informe.md` deja marcados como pendientes los demás verdaderos positivos.

Ampliación

Si has terminado, elige una:

1. **Línea base.** En un proyecto grande no se pueden corregir cuatrocientos avisos de golpe, pero sí se puede impedir que aparezcan avisos nuevos. Con el código original, genera un informe JSON y úsalo como línea base:

```
bandit -r reservas -f json -o base.json
bandit -r reservas -b base.json
echo $?
```

Bandit no muestra los avisos que ya estaban en `base.json`. Añade después una función nueva con un fallo (por ejemplo, una que use `eval`), repite las dos últimas órdenes y comprueba que solo te avisa de esa. Explica qué pasa con el código de salida en cada caso y por qué eso es útil en un proyecto grande.

1. **Opciones del ping.** Aunque quites `shell=True`, si lo que llega empieza por `-` el programa `ping` puede interpretarlo como una opción. Explica por qué tu validación con `ipaddress` lo evita, y escribe una prueba que lo demuestre.

13. Preguntas

Contesta con tus palabras en `informe.md`.

1. ¿Por qué `bandit -r .` daba tantos avisos más que `bandit -r reservas`? ¿Qué problema crea un informe con cientos de avisos?
2. ¿Qué diferencia hay entre severidad y confianza? Pon un ejemplo de esta práctica con severidad alta y otro con confianza baja.
3. ¿Qué verdadero positivo desaparece con `-ll -ii`? ¿Qué te dice eso sobre filtrar por confianza?
4. Explica uno de tus falsos positivos: qué vio Bandit y por qué en este código no es un fallo.
5. ¿Por qué `assert` no sirve como control de acceso?
6. ¿Qué fallo no detectó Bandit? ¿Por qué crees que no puede detectarlo?

14. Qué tienes que entregar

Entrega una carpeta `NN-PPS-bandit` (NN es tu número de lista) comprimida en ZIP con:

1. El proyecto corregido entero, **sin la carpeta** `.venv`, con tu `tests/test_regresion.py` y la configuración de Bandit en `pyproject.toml`.
2. `bandit-antes.txt` y `bandit-despues.txt`.
3. `salida-pytest.txt`, con la salida de `python -m pytest -v > salida-pytest.txt`.
4. `informe.md` completo: la tabla de triaje, las correcciones, el fallo que Bandit no vio y las respuestas.

Para crear el ZIP:

```
cd ~/pps
zip -r NN-PPS-bandit.zip PPS-2026-11-10-bandit-LAB \
    -x "*/.venv/*" "*/__pycache__/*" "*/.pytest_cache/*"
```

Si no te ha dado tiempo a terminar, entrega lo que tengas y explica en `informe.md` en qué punto exacto te has quedado.

15. Antes de entregar, comprueba que

1. La tabla de triaje tiene una fila por cada aviso del informe inicial, con veredicto y explicación.
2. Cada verdadero positivo tiene su prueba de regresión, y esa prueba fallaba con el código original.
3. `python -m pytest` termina sin ninguna prueba en rojo.
4. `bandit -c pyproject.toml -r .` no da ningún aviso sin justificar. Lo normal es que termine sin avisos y `echo $?` dé `0`, porque los que quedan están corregidos o marcados con un `# nosec` explicado.
5. Cada `# nosec` lleva identificador y un comentario que lo justifica, y ninguno tapa un verdadero positivo.
6. El fallo de `informes.py` está corregido y probado.
7. El ZIP no lleva `.venv`.

16. Un apunte de seguridad

Las herramientas como Bandit se llaman de análisis estático, o SAST, porque leen el código sin ejecutarlo. Son baratas y rápidas, y por eso se ponen en todos los proyectos serios. Pero un informe de Bandit a cero no significa que el código sea seguro, igual que un 100 % de cobertura no significaba que las pruebas fueran buenas. Lo que sí significa, si el triaje está bien hecho, es que los errores más conocidos no están, y que cuando alguien añada uno nuevo te vas a enterar.

El código de la práctica tiene fallos a propósito y las contraseñas son inventadas. No lo despliegues en ningún servidor ni copies esas funciones en otro proyecto sin corregirlas.

Referencias

- Documentación de Bandit: https://bandit.readthedocs.io/
- Lista de reglas de Bandit: https://bandit.readthedocs.io/en/latest/plugins/index.html
- Configuración en `pyproject.toml`: https://bandit.readthedocs.io/en/latest/config.html
- Catálogo CWE de MITRE: https://cwe.mitre.org/
- Módulo `secrets`: https://docs.python.org/es/3/library/secrets.html

- `hashlib.scrypt`: https://docs.python.org/es/3/library/hashlib.html
- OWASP, Path Traversal: https://owasp.org/www-community/attacks/Path_Traversal
- OWASP, Password Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html